Quiz — 1.1.2 Types of Processor — ANSWER SHEET

Separate answer sheet / mark scheme — keep for marking.

Answer key

Section A (8 marks — 1 each)

1. **B** — Von Neumann uses one shared memory and bus for both data and instructions.
2. **C** — Harvard has separate memories/buses, allowing simultaneous instruction fetch and data access.
3. **B** — RISC = few, simple, fixed-length instructions, aiming for one cycle each.
4. **B** — In CISC the complexity is in the hardware (microcode); in RISC it is in software/compiler.
5. **B** — RISC/ARM is used in mobile/embedded devices for low power consumption.
6. **B** — GPUs apply the same operation to large data blocks in parallel (data parallelism).
7. **B** — Training a neural network is a non-graphics (GPGPU) use; rendering pixels is standard graphics.
8. **B** — Data dependency (a step needs the previous step's result) prevents effective parallelisation.

Section B (12 marks)

9. **[3]** — 1 mark per valid difference (max 3). Examples: - Von Neumann = one shared memory; Harvard = separate memories for data and instructions. - Von Neumann = shared bus; Harvard = separate buses. - Von Neumann can suffer a bottleneck (cannot fetch instruction and access data at once); Harvard can do both simultaneously. - Von Neumann is simpler/cheaper/more flexible; Harvard is more complex/costly. - Von Neumann used in general-purpose PCs; Harvard used in embedded/microcontrollers/DSPs.

10. **[2]** — 1 mark: it uses a **single unified main memory** (Von Neumann style) for programs and data; 1 mark: but has **separate L1 instruction and data caches** (Harvard style) close to the core, giving some simultaneous-access benefit.

11. **[2]** — 1 mark per difference (max 2). Examples: CISC has many complex variable-length instructions vs RISC few simple fixed-length; complexity in hardware (CISC) vs software/compiler (RISC); RISC easier to pipeline; RISC lower power; CISC used in x86 desktops vs RISC in ARM mobile/embedded.

12. **[2]** — 1 mark: a GPU has **thousands of small cores**; 1 mark: it can process many pixels/vertices **in parallel** (the same operation on large data sets simultaneously), so rendering is much faster than doing it sequentially.

13. **[1]** — A processor (chip) containing **two or more independent cores**, each able to run its own fetch–decode–execute cycle, allowing simultaneous processing.

14. **[2]** — 1 mark: there is a **data dependency** — the search needs the completed (sorted) list, so it cannot begin until the sort finishes. 1 mark: the two stages must run **sequentially**, so extra cores cannot speed up the overall sequence (only the sort itself might be parallelised); benefit is limited by the **serial fraction**.

Section C (5 marks)

15. **[5]** — Point/level marking, up to 5 for a justified, applied discussion. Indicative content: - The workload applies the **same calculation to millions of data points** — this is highly **data-parallel**, with little branching or dependency between points. - A **GPU** has thousands of small cores and excels at performing the same operation on large data sets in parallel (data parallelism / GPGPU), so it can process far more data points simultaneously. - A multicore CPU has fewer, more powerful cores better suited to sequential, branch-heavy, dependency-laden tasks — fewer parallel lanes for this kind of work. - Caveat: the software must be written to use the GPU; very branchy or strongly dependent calculations would suit the CPU better. - Justified conclusion: for this uniform, massively data-parallel simulation the **GPU is the better choice** because it maximises throughput across many data points. Max 4 if no clear justified judgement is given.

Total: 8 + 12 + 5 = 25